COL100: Introduction to Computer Science
Lab 13: Object-Oriented Programming

April 15, 2025

1 Introduction

File handling is an essential part of programming that allows you to create, read, write, and manage files. Python provides built-in functions and libraries to perform file operations efficiently. In this tutorial, we will explore the importance of file handling, its different methods, and how to work with various file types such as text files (`.txt`) and CSV files (`.csv`).

1.1 Importance of File Handling in Python

Files are used to store data, and file handling makes it possible to:

1. **Persist Data:** Store data permanently for later use.
2. **Exchange Information:** Allow for the transfer of data between systems or users.
3. **Automate Tasks:** Process files programmatically without manual intervention.
4. **Analyze Large Datasets:** Work with files containing large amounts of data, such as CSV files.

1.2 File Operations in Python

Python provides the `open()` function to handle file operations. The file can be opened in various modes:

- **'r' (Read Mode):** Opens a file for reading.
- **'w' (Write Mode):** Opens a file for writing (overwriting if the file exists).
- **'a' (Append Mode):** Opens a file for writing but appends to the file if it exists.
- **'r+' (Read and Write Mode):** Opens a file for both reading and writing.
- **Binary Modes ('b'):** Used for non-text files (e.g., images).

1.3 File Handling with Text Files (`.txt`)

1.3.1 Writing to a Text File

```
# Open a file in write mode
file = open("example.txt", "w")

# Write content to the file
file.write("Hello, World!\nThis is a text file.")

# Close the file
file.close()

print("File written successfully!")
```

Listing 1: Writing to a text file

1.3.2 Appending Data to a Text File

```
# Open the file in append mode
file = open("example.txt", "a")

# Append content to the file
file.write("\nAppending new content here.")

# Close the file
file.close()

print("Content appended successfully!")
```

Listing 2: Appending to a text file

1.3.3 Reading from a Text File

```
# Open the file in read mode
file = open("example.txt", "r")

# Read the entire content
content = file.read()

# Print the content
print(content)
```

```
9
10 # Close the file
11 file.close()
```

Listing 3: Reading the entire content of a text file

1.3.4 Reading Line by Line

```
1 # Open the file
2 with open("example.txt", "r") as file:
3     for line in file:
4         print(line.strip())  # Removes trailing newline
    characters
```

Listing 4: Reading a text file line by line

1.4 File Handling with CSV Files

CSV (Comma-Separated Values) files are widely used to store tabular data. Python provides the `csv` module for working with these files.

1.4.1 Writing to a CSV File

```
1 import csv
2
3 # Open a file in write mode
4 with open("data.csv", "w", newline="") as csvfile:
5     writer = csv.writer(csvfile)
6
7     # Writing the header row
8     writer.writerow(["Name", "Age", "City"])
9
10    # Writing data rows
11    writer.writerow(["Alice", 25, "New York"])
12    writer.writerow(["Bob", 30, "Los Angeles"])
13
14 print("CSV file written successfully!")
```

Listing 5: Writing to a CSV file

1.4.2 Reading a CSV File

```
import csv

# Open the CSV file in read mode
with open("data.csv", "r") as csvfile:
    reader = csv.reader(csvfile)

    # Print each row
    for row in reader:
        print(row)
```

Listing 6: Reading from a CSV file

1.4.3 Working with CSV Files as Dictionaries

```
import csv

# Writing into CSV file as dictionaries
with open("data_dict.csv", "w", newline="") as csvfile:
    fieldnames = ["Name", "Age", "City"]
    writer = csv.DictWriter(csvfile, fieldnames=fieldnames)

    # Writing the header
    writer.writeheader()

    # Writing rows
    writer.writerow({"Name": "Alice", "Age": 25, "City": "
   New York"})
    writer.writerow({"Name": "Bob", "Age": 30, "City": "Los
   Angeles"})

# Reading CSV file as dictionaries
with open("data_dict.csv", "r") as csvfile:
    reader = csv.DictReader(csvfile)

    for row in reader:
        print(row)
```

Listing 7: Working with CSV files using DictWriter and DictReader

1.5 Best Practices for File Handling

- **Use Context Managers:** Always use `with open()` to ensure files are closed automatically after use.
- **Handle Exceptions:** Use `try-except` blocks to handle errors (e.g., file not found).
- **Specify Correct File Paths:** Ensure you use absolute or relative paths correctly.
- **Choose the Right Mode:** Use append mode (`'a'`) to add data without overwriting and write mode (`'w'`) for creating new files.
- **Test File Encoding:** For non-English text, specify the encoding (e.g., `open("file.txt", "r", encoding="utf-8")`).

1.6 Summary of Use Cases

- **Text Files (`.txt`):**
  - Use for simple data storage, logs, or configuration files.
  - Great for lightweight data that doesn't require tabular formatting.
- **CSV Files (`.csv`):**
  - Use for structured data like tables or datasets.
  - Ideal for database exports, spreadsheet integrations, or handling large amounts of tabular data.

1.7 Conclusion

By mastering file handling in Python, you can efficiently interact with data stored in files, automate tasks, and handle large datasets. Use the examples above as a reference for your projects!

2 Submission Problem

2.1 Analyse Data

A random generator is used to create data for a student and activity management system (SAMS). This file is provided to you as a CSV file `generated_data.csv` with the following structure:

```
idi,name,birth_year,birth_month,birth_day,gender,registration_id,
    athletic_score,leadership_score,talent_score,gpa,class_assigned,
    selected_activity,grade_level,talent,olympiad_scores,
    subject_specialization,fitness_score,sports_category,subject,
    mentor_grade,mentor_class,judge
```

This data file will contain 5036 lines. The first line follows the header stated earlier, the next 5000 representing users, and the last 35 teachers. Your task is to write a Python program that reads this file, processes the data, and find `class_assigned` where there isn't a teacher assigned or teacher who is a judge. You need to output this data into a new file called `errors.txt`, in the following format:

```
Class [mentor_class/mentor_grade]: Error [TEACHER/JUDGE] not found.
```

Note:

- There are classes from 6 to 12 in the school.
- A judge needs to be selected amongst teachers from the same class. Errors should be shown if no teacher within a class (say 9) has `judge` attribute as true.
- In the case where both errors are found, first print the teacher and then the judge.
- The order of class_assigned should be ascending order. That is `class_assigned` should be 6A, 7B, 7C...

2.1.1 Sample Input

A file `generated_data.csv`.

2.1.2 Sample Output

A file `errors.csv` with data like:

```
Class 9C: Error TEACHER not found.
Class 10: Error JUDGE not found.
```